

Topic: Applications of Stack – Well-Formedness of Parentheses

Q. Explain how a stack can be used to check the well-formedness of parentheses in an expression. Write a step-by-step algorithmic approach.

📌 **Definition to Remember**

An expression is **well-formed (balanced)** if every opening bracket (`(` , `{` , `[` has a corresponding, correctly-ordered closing bracket. A **Stack** is used because its LIFO property ensures the most recently opened bracket is always checked first when a closing bracket is encountered.

1. Why Stack Works for This Problem

- LIFO ensures the **most recently opened bracket** is at the top of the stack.
- When a closing bracket arrives, it must match the bracket most recently pushed — exactly what `top()` provides.

2. Algorithm

```
1. Initialize an empty stack.
2. Scan expression left to right, character by character.
3. If character is an OPENING bracket → (, {, [:
   Push it onto the stack.
4. If character is a CLOSING bracket → ), }, ]:
   a. If stack is EMPTY → return FALSE (extra closing bracket).
   b. Pop the top element.
   c. If popped bracket ≠ matching opening → return FALSE (mismatch).
5. After full scan:
   If stack is EMPTY → return TRUE (Well-Formed).
   Else → return FALSE (unclosed opening brackets).
```

3. Trace Example: { [()] }

Character	Action	Stack State
{	Push {	{
[	Push [	{ [
(	Push (	{ [(
)	Pop (— matches) ✓	{ [
]	Pop [— matches] ✓	{
}	Pop { — matches } ✓	(Empty)
End	Stack is empty → VALID ✓	

Invalid Example: { (] }

* Read] → Pop (— **mismatch!** → return **FALSE**.

★ **Must-Write Points (for 10 marks)**

1. Stack is ideal for this because of its **LIFO** property — last opened = first to match.
2. Push all **opening brackets** (, {, [onto the stack.
3. For each **closing bracket**: if stack is empty → FALSE; else pop and compare.
4. If popped bracket doesn't match closing bracket → FALSE (mismatched pairs).
5. After full scan: if stack is **empty** → **TRUE (balanced)**; non-empty → FALSE (unclosed).
6. Time complexity: **O(n)** — each character is processed once.
7. Application: compilers, parsers, and IDE bracket highlighting use this algorithm.

⚡ **Quick Recall**

Opening → Push → Closing → Stack Empty? (FALSE) → Pop → Match? (FALSE if not) → After scan: Stack Empty=TRUE, Non-empty=FALSE → O(n)